



Filipe Gouveia

(py)ModRev – Repairing inconsistent Boolean logical models of biological regulatory networks

LASIGE, Faculdade de Ciências, Universidade de Lisboa, Portugal

BioLogics Workshop

Marseille 2026

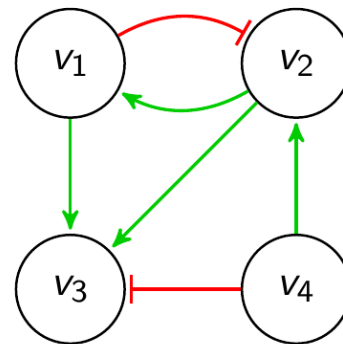


Boolean logical model of regulatory networks

A regulatory network is a collection of molecular compounds that interact with each other.

Boolean logical model:

- Compounds represented by a Boolean variable
- Interaction defined as positive (activation) or negative (inhibition)
- Regulations defined as Boolean functions



$$f_{v_1} = v_2$$

$$f_{v_2} = \neg v_1 \wedge v_4$$

$$f_{v_3} = v_1 \vee (v_2 \wedge \neg v_4)$$

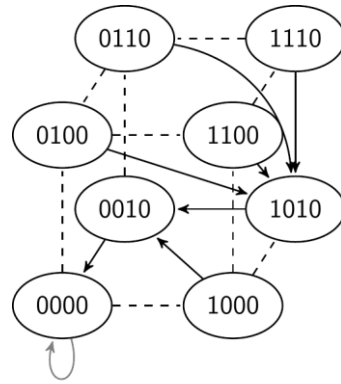
Dynamics

The value of each node can change in time defining the state of the network.

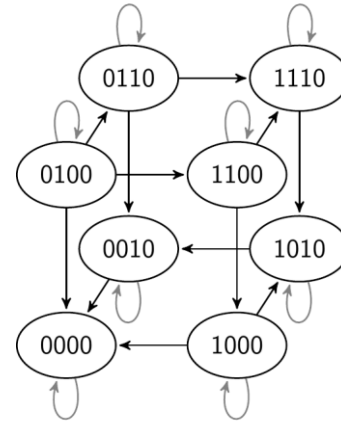
The regulatory functions update the value of the corresponding node.

Different update schemes can be considered, defining State Transition Graphs.

Synchronous



Asynchronous



Model Revision

As new experimental data becomes available, models may become **inconsistent**!

- Not being able to reproduce the new information.
- Models need to be **revised**.

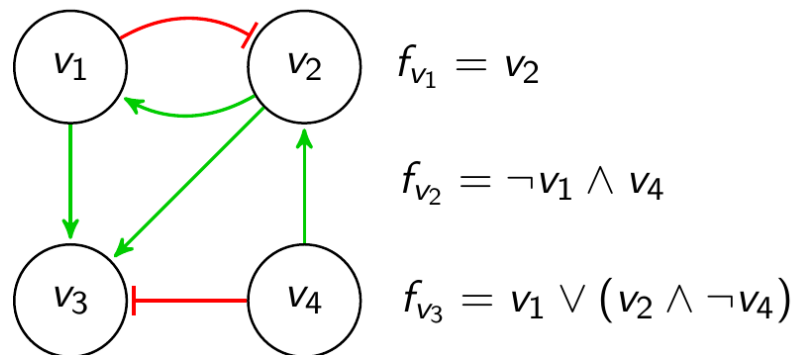
A model is **consistent** if all its nodes are consistent.

- Value of each node given by its regulatory function is equal to the observed value.

A model is **inconsistent** otherwise.

How can we repair an inconsistent model?

- Change a regulatory function?
 - 2^{2^k} possibilities for each node!
- Change the type of interactions?
- Add or remove interactions?



There are $\approx 10^{24}$ possible combinations!
(65536 Boolean functions with 4 regulators)

Repair Operations

Cause	Repair Operation
Wrong regulatory function	Function change
Wrong interaction type	Edge sign flip
Wrong regulator	Remove edge
Missing regulator	Add edge

Optimization Criteria:

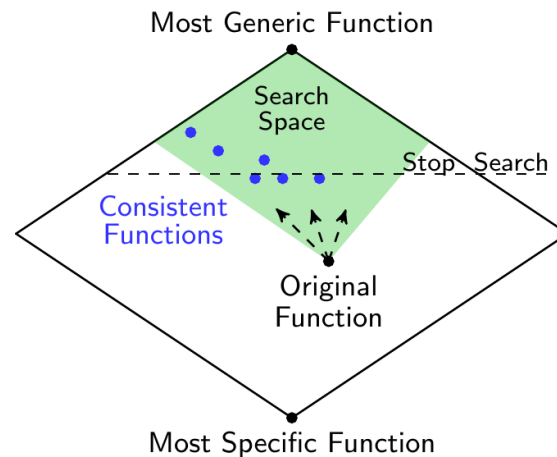
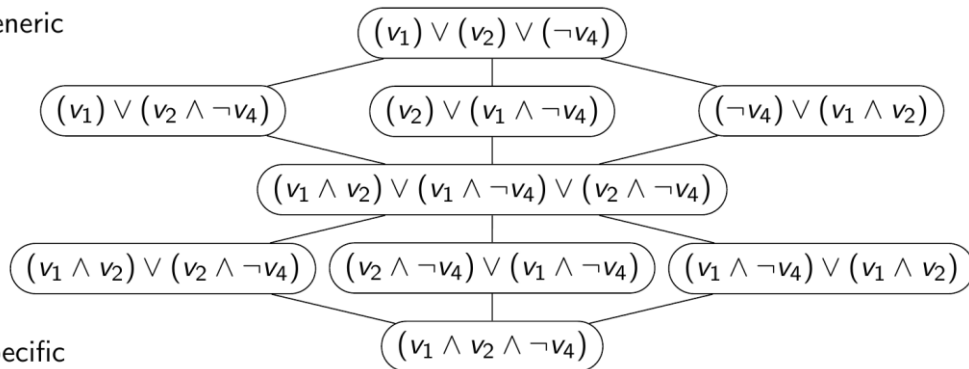
1. Minimize interaction addition/removal.
2. Minimize interaction type changes.
3. Minimize Boolean function changes.

Function Repair

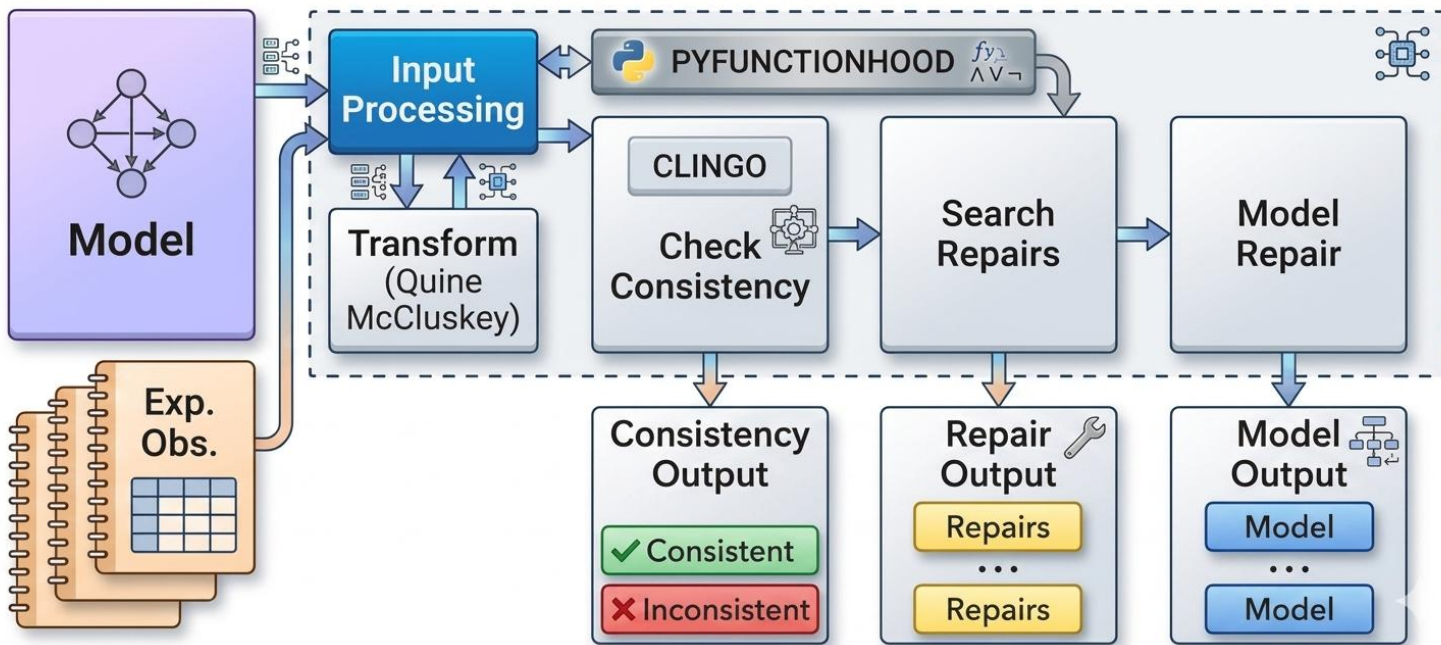
We consider monotone non-degenerate Boolean functions.

more generic

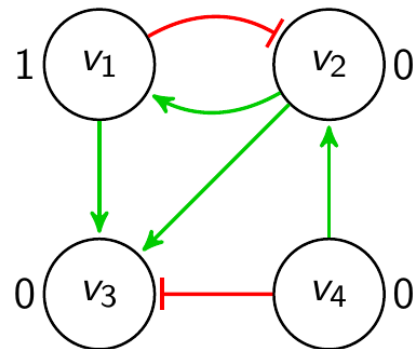
more specific



Overview



Model Repair Example (Stable State)



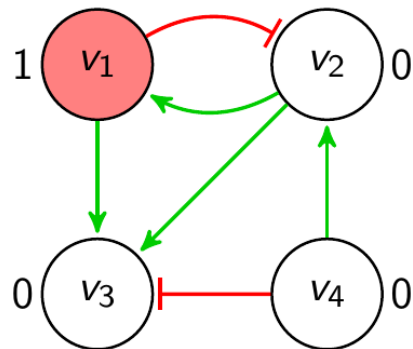
$$f_{v_1} = v_2$$

$$f_{v_2} = \neg v_1 \wedge v_4$$

$$f_{v_3} = v_1 \vee (v_2 \wedge \neg v_4)$$

- Inconsistent model.
- Node v_1 is inconsistent.
- Change the interaction between v_2 and v_1 .
- Node v_3 is inconsistent.
- Consistent model!

Model Repair Example (Stable State)



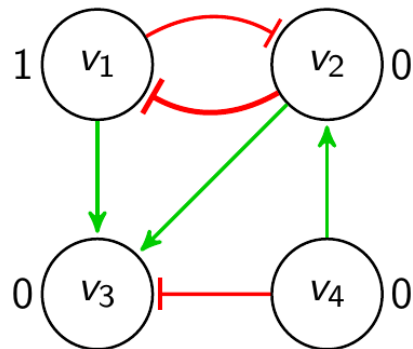
$$f_{v_1} = v_2$$

$$f_{v_2} = \neg v_1 \wedge v_4$$

$$f_{v_3} = v_1 \vee (v_2 \wedge \neg v_4)$$

- Inconsistent model.
- Node v_1 is inconsistent.
- Change the interaction between v_2 and v_1 .
- Node v_3 is inconsistent.
- Consistent model!

Model Repair Example (Stable State)



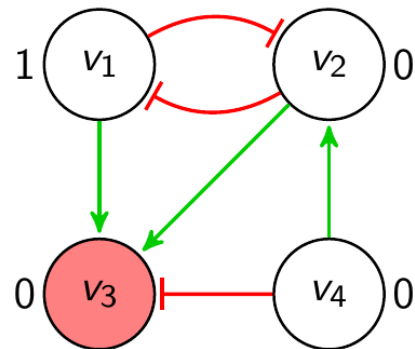
$$f_{v_1} = \neg v_2$$

$$f_{v_2} = \neg v_1 \wedge v_4$$

$$f_{v_3} = v_1 \vee (v_2 \wedge \neg v_4)$$

- Inconsistent model.
- Node v_1 is inconsistent.
- Change the interaction between v_2 and v_1 .
- Node v_3 is inconsistent.
- Consistent model!

Model Repair Example (Stable State)



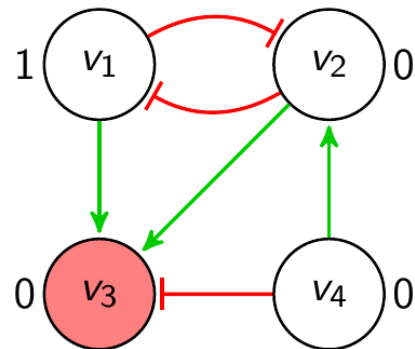
$$f_{v_1} = \neg v_2$$

$$f_{v_2} = \neg v_1 \wedge v_4$$

$$f_{v_3} = v_1 \vee (v_2 \wedge \neg v_4)$$

- Inconsistent model.
- Node v_1 is inconsistent.
- Change the interaction between v_2 and v_1 .
- Node v_3 is inconsistent.
- Consistent model!

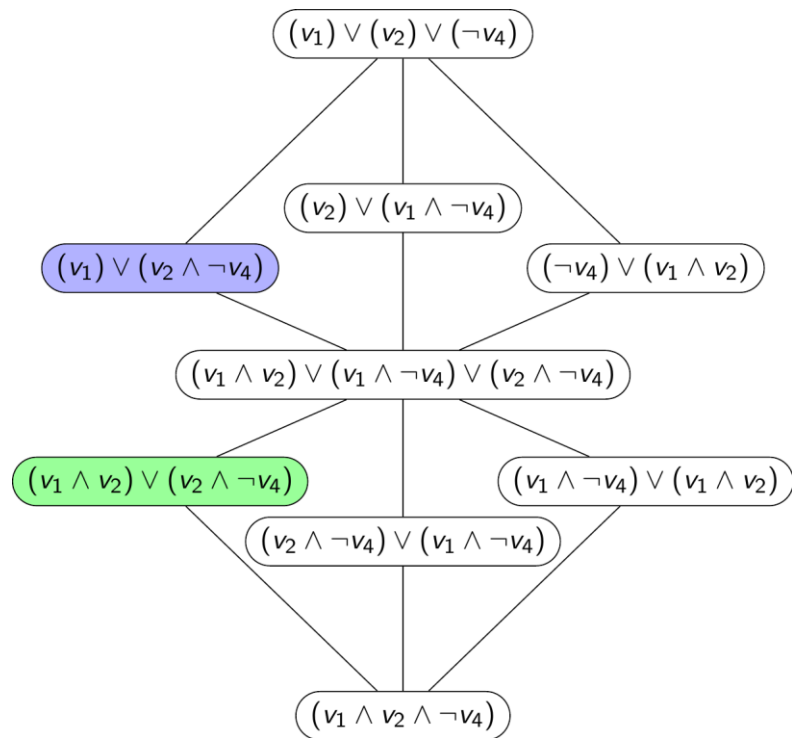
Model Repair Example (Stable State)



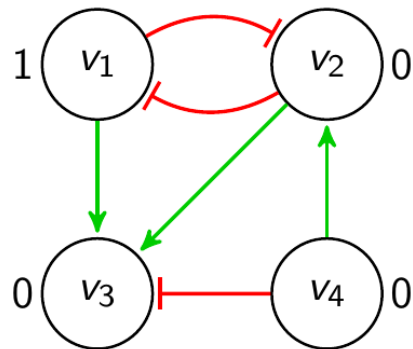
$$f_{v_1} = \neg v_2$$

$$f_{v_2} = \neg v_1 \wedge v_4$$

$$f_{v_3} = v_1 \vee (v_2 \wedge \neg v_4)$$



Model Repair Example (Stable State)



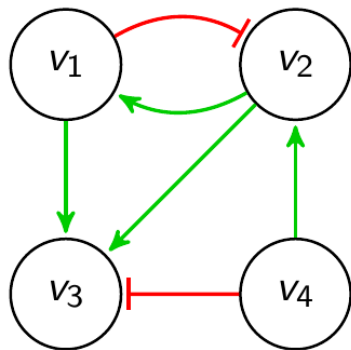
$$f_{v_1} = \neg v_2$$

$$f_{v_2} = \neg v_1 \wedge v_4$$

$$f_{v_3} = (v_1 \wedge v_2) \vee (v_2 \wedge \neg v_4)$$

- Inconsistent model.
- Node v_1 is inconsistent.
- Change the interaction between v_2 and v_1 .
- Node v_3 is inconsistent.
- Consistent model!

Model Repair Example (Time-series synchronous)



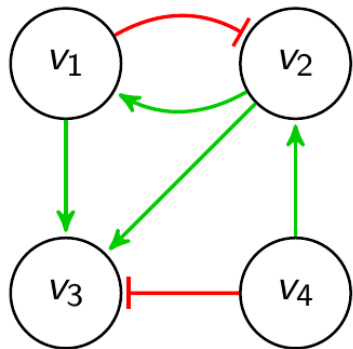
$$f_{v_1} = v_2$$

$$f_{v_2} = \neg v_1 \wedge v_4$$

$$f_{v_3} = v_1 \vee (v_2 \wedge \neg v_4)$$

		Time		
		t_0	t_1	t_2
Node	v_1	1	1	0
	v_2	1	0	1
	v_3	0	1	0
	v_4	1	1	1

Model Repair Example (Time-series synchronous)



$$f_{v_1} = v_2$$

$$f_{v_2} = \neg v_1 \wedge v_4$$

$$f_{v_3} = v_1 \vee (v_2 \wedge \neg v_4)$$

		Time		
		t_0	t_1	t_2
Node	v_1	1	1	0
	v_2	1	0	1
	v_3	0	1	0
	v_4	1	1	1

ModRev and pyModRev – What can we do?

- Confront models with different experimental observations simultaneously.
 - Stable state and time-series.
- Different update schemes (simultaneously):
 - Synchronous, asynchronous and complete.
- Support missing values in observations.
 - Minimize number of inconsistent nodes.
- Avoid given state as steady state.
- Produce optimal repairs.



ModRev (C++)



pyModRev (python)

Next Steps

- Support different update schemes.
 - Most Permissive
- Full support to reachability properties.
 - Go from fixed to variable time steps
 - Check reachability
 - Avoid reaching some state
- Improve function repair search efficiency.

Thank you!



pyModRev